

IEEE-CIS Financial Fraud Detection Engine

End-to-End System, Pipeline & Production Deployment Architecture Specification

1. High-Level Architecture Overview

The IEEE-CIS Fraud Detection Engine is built on a **Disk-Cached, Memory-Optimized Streaming ETL & Machine Learning Architecture**. It processes over 1.1 million financial transactions (1.3 GB raw CSVs) under strict physical RAM constraints (< 1.5 GB RAM cap) and delivers real-time fraud probability scores with an Out-of-Fold (OOF) ROC-AUC of **0.9421**.

SYSTEM COMPONENT FLOW:

- [1. Data Layer] Raw CSV Files (train/test transaction & identity tables)
■
- [2. Streaming Ingestion] Chunked Reader (50k rows/batch) + Type Downcaster (float32/int8)
■
- [3. ETL & Disk Cache] Isolated Train/Test Phase Pipeline ■■> Joblib Binary Disk Cache
■
- [4. Feature Pipeline] Time Extractor + Log Scaler + Categorical Encoder + Sparse Pruning (437 -> 278 cols)
■
- [5. Model Engine] 3-Fold Stratified K-Fold CV + Low-Level xgb.DMatrix + tree_method='hist'
■
- [6. Inference & Serve] Chunked Batch Prediction Engine (100k rows/batch) ■■> submission.csv / REST API

2. Detailed Component Specifications

Component	Architectural Responsibility	Tech Stack & Specification
Ingestion & Type Downcasting	Reads raw CSV chunks, downcasts float64 -> float32 and int64 -> int8/int16 based on min/max bounds.	Pandas <code>`read_csv(chunksize=50000)`</code> , <code>`np.iinfo`</code> , <code>`np.finfo`</code> downcasting
Disk-Cache Storage	Decouples train and test ingestion so they never occupy RAM simultaneously. Flushes memory via <code>`gc.collect()`</code> .	<code>`joblib.dump(compress=1)`</code> storing binary <code>`pkl`</code> files in <code>`_temp_cache/`</code>
Feature Engineering	Extracts DT_M, DT_W, DT_D, DT_hour, DT_day_of_week from TransactionDT; applies log1p scaling to transaction amounts.	NumPy vectorized math, <code>`LabelEncoder`</code> fitted over combined vocabularies
Feature Pruner	Filters out 159 sparse Vesta (V) features with missing rate >= 50%, reducing feature dimension from 437 to 278.	Pandas <code>`isnull().mean()`</code> missingness analysis (Matrix: 626 MB)
Model Training Engine	Trains 3-Fold Stratified K-Fold CV using low-level <code>`xgb.DMatrix`</code> and histogram-based binning (<code>`max_bin=128`</code>).	XGBoost 3.2.0 C++ API (<code>`tree_method='hist'`</code> , <code>`max_depth=6`</code> , <code>`n_jobs=2`</code>)
Inference Pipeline	Loads serialized JSON booster models and computes ensemble predictions in 100k row batches.	<code>`xgb.Booster.load_model()`</code> , vectorised chunk prediction

3. Memory Optimization & Zero-OOM Strategy

Standard machine learning pipelines fail on 8 GB RAM machines due to simultaneous memory residency of train DataFrames (1.0 GB), test DataFrames (0.9 GB), and intermediate pandas/C++ consolidation buffers (1.2 GB). Our architecture solves this through four strict execution layers:

Optimization Layer	Standard Approach	Our Architecture	RAM Saved
Pipeline Coupling	Train & Test in RAM together	Phase 1 Train -> Disk Cache -> Free RAM -> Phase 2 Test	1.0 GB
Data Downcasting	Default float64 / int64	Aggressive downcasting to float32 / int16 / int8	450 MB
Feature Pruning	All 437 features in RAM	Pruned 159 sparse V columns (>= 50% null)	400 MB
Matrix & DMatrix API	Pandas DataFrame -> High-level fit()	C-contiguous NumPy float32 -> low-level <code>`xgb.DMatrix`</code>	1.2 GB (C++ buffer)

4. Model & Cross-Validation Architecture

Cross-Validation Scheme: 3-Fold Stratified K-Fold. Preserves 3.50% fraud target ratio in every fold.

XGBoost Hyperparameters: objective='binary:logistic', eval_metric='auc', learning_rate=0.05, max_depth=6, max_bin=128, subsample=0.8, colsample_bytree=0.8, tree_method='hist', n_jobs=2.

Model Serialization: Fold models stored as `xgb\_model\_fold\_0.json`, `xgb\_model\_fold\_1.json`, `xgb\_model\_fold\_2.json`.

Fold ID	Validation Set Size	Validation ROC-AUC	Early Stop Iteration
Fold 1	196,847 rows	0.94442	Iteration 499
Fold 2	196,847 rows	0.93988	Iteration 499
Fold 3	196,846 rows	0.94208	Iteration 499
Overall OOF	590,540 rows	0.94208 ± 0.00185	3.84 Mins Total

5. Production & Cloud Deployment Architecture

PRODUCTION MICROSERVICE DESIGN (Low Latency < 5ms):

```
[ Client / Gateway ] HTTP POST Request (Transaction Payload)
  ■
[ FastAPI Container ] Dockerized microservice endpoint (`/predict`)
  ■■■> [ Redis Cache ] O(1) fetch for pre-computed user profile & aggregate features
  ■■■> [ TreeLite / ONNX Engine ] Compiled native C decision tree model execution
  ■■■> [ Risk Decision Engine ] Score Thresholding:
    • Score < 0.15 ■■■> APPROVE (Green)
    • 0.15 <= Score < 0.60 ■■■> STEP-UP 2FA / MANUAL REVIEW (Yellow)
    • Score >= 0.60 ■■■> DECLINE / BLOCK (Red)
```